

VNUHCM – UNIVERSITY OF SCIENCE  
FACULTY OF DATA SCIENCE  
DEPARTMENT OF COMPUTER SCIENCE

[https://upload.wikimedia.org/wikipedia/commons/f/f8/Logo\\_1](https://upload.wikimedia.org/wikipedia/commons/f/f8/Logo_1)

ACADEMIC COURSE REPORT  
**KNOWLEDGE  
REPRESENTATION**

**HLG LLM UTILS: AN ADVANCED GRAPH RAG  
FRAMEWORK WITH MULTI-AGENT  
ORCHESTRATION, HIERARCHICAL LEIDEN  
COMMUNITY DETECTION, AND CYPHER QUERY  
SELF-CORRECTION**

**Instructor:** Assoc. Prof. Dr. Do Van Nhon

**Students:** 1. Nguyen Hoai Nam – Student ID: 25C01014  
2. Pham Cung Le Nhan Vu – Student ID: 25C01049

**Major:** Data Science

**Academic Intake:** K35

## Executive Abstract

Standard Retrieval-Augmented Generation (Naive RAG) systems based on flat chunking and vector space similarity (e.g., Cosine Similarity over high-dimensional embeddings) suffer from two fundamental bottlenecks: (1) **Global Context Fragmentation**, causing failures in answering corpus-wide macro-summarization queries; and (2) **Multi-Hop Reasoning Failures**, which prevent LLMs from traversing complex dependency chains across disparate documents. This report presents **HLG LLM Utils**, a modular, production-grade Graph RAG framework built on a Python monorepo architecture (`uv workspaces`).

Our framework introduces key contributions in Knowledge Representation and Information Retrieval:

1. **Formal Property Knowledge Graph Modeling:** Automated extraction of entities ( $\mathcal{E}$ ), relationships ( $\mathcal{R}$ ), and attributes from unstructured text, persisting them onto a Neo4j Property Knowledge Graph.
2. **Hierarchical Leiden Community Detection:** Partitioning the entity graph into multi-level community hierarchies, combined with parallel Map-Reduce LLM summarization for Global Search queries.
3. **Cypher Query Self-Correction State Machine:** A LangGraph state machine translating natural language queries into executable Cypher statements, featuring a 3-iteration error feedback loop against Neo4j execution logs.
4. **ARK Autonomous Multi-Agent Exploration:** Implementing the ACL 2026 Adaptive Retriever of Knowledge algorithm to dynamically adjust breadth-depth graph traversal with Voting Rank Fusion (VRF).

Empirical evaluations on benchmark datasets using RAGAS metrics show that HLG LLM Utils achieves 0.94 Faithfulness, 0.89 Context Precision, and a 96.8% Cypher execution success rate, substantially outperforming standard Naive RAG baselines.

**Keywords:** Graph RAG, Knowledge Representation, Hierarchical Leiden Algorithm, Cypher Self-Correction, Multi-Agent Systems, LangGraph, FastMCP.

# Contents

|                                                                     |           |
|---------------------------------------------------------------------|-----------|
| <b>Executive Abstract</b>                                           | <b>i</b>  |
| <b>1 Introduction &amp; Problem Statement</b>                       | <b>1</b>  |
| 1.1 Background & Motivation . . . . .                               | 1         |
| 1.2 Limitations of Naive Flat RAG . . . . .                         | 1         |
| 1.3 Research Objectives & Core Contributions . . . . .              | 1         |
| <b>2 Theoretical Foundations &amp; Related Work</b>                 | <b>3</b>  |
| 2.1 Knowledge Representation Paradigms . . . . .                    | 3         |
| 2.1.1 From Semantic Networks to Property Knowledge Graphs . . . . . | 3         |
| 2.2 Vector Space Embeddings & Hybrid Retrieval . . . . .            | 3         |
| 2.3 Community Detection & The Leiden Algorithm . . . . .            | 3         |
| 2.4 Microsoft GraphRAG & ARK Exploration . . . . .                  | 4         |
| <b>3 Formal Knowledge Representation &amp; Schema Design</b>        | <b>5</b>  |
| 3.1 Property Knowledge Graph Formalization . . . . .                | 5         |
| 3.2 Neo4j Graph Schema Specification . . . . .                      | 5         |
| <b>4 System Architecture &amp; Monorepo Design</b>                  | <b>6</b>  |
| 4.1 UV Workspace Monorepo Architecture . . . . .                    | 6         |
| 4.2 Backend Services & FastAPI Integration . . . . .                | 6         |
| <b>5 Advanced Search &amp; Inference Algorithms</b>                 | <b>7</b>  |
| 5.1 Hierarchical Leiden Community Detection . . . . .               | 7         |
| 5.2 Cypher Self-Correction State Machine . . . . .                  | 7         |
| <b>6 Experimental Evaluation &amp; Benchmark Results</b>            | <b>9</b>  |
| 6.1 Experimental Setup & RAGAS Benchmarks . . . . .                 | 9         |
| <b>7 Conclusion &amp; Task Division</b>                             | <b>11</b> |
| 7.1 Conclusion . . . . .                                            | 11        |
| 7.2 Student Contribution Matrix . . . . .                           | 11        |
| <b>References</b>                                                   | <b>12</b> |

# List of Figures

|     |                                                                      |   |
|-----|----------------------------------------------------------------------|---|
| 4.1 | Package Dependencies in the UV Workspace Architecture . . . . .      | 6 |
| 5.1 | LangGraph State Machine for Self-Correcting Cypher Queries . . . . . | 8 |

# List of Tables

|     |                                                           |    |
|-----|-----------------------------------------------------------|----|
| 3.1 | Vertex Attribute Specifications in Neo4j Schema . . . . . | 5  |
| 6.1 | Quantitative Performance Benchmark Comparison . . . . .   | 10 |
| 7.1 | Task Division Matrix Between Group Members . . . . .      | 11 |

## Chapter 1

# Introduction & Problem Statement

## 1.1 Background & Motivation

In the era of Large Language Models (LLMs), natural language understanding and generation have reached unprecedented levels. However, standalone LLMs remain susceptible to hallucinations and knowledge cutoff constraints because their knowledge is statically encoded within model parameters. Retrieval-Augmented Generation (RAG) emerged as the dominant paradigm to ground LLM responses in external authoritative data sources.

Despite its initial success, Naive RAG—which relies on sliding-window text chunking and vector space cosine similarity—exhibits severe structural limitations when applied to complex enterprise Knowledge Representation tasks.

## 1.2 Limitations of Naive Flat RAG

Through theoretical analysis and empirical observations, we identify three critical failure modes of Naive RAG:

1. **Structural Context Fragmentation:** Fixed-size chunking (e.g., 1,000 tokens) severs conceptual relationships spanning across different paragraphs or separate documents.
2. **Multi-Hop Reasoning Blindspots:** When answering queries requiring multi-step relational chains ( $A \rightarrow B \rightarrow C$ ), vector similarity retrieves chunks mentioning  $A$  or  $C$ , but misses the crucial intermediate link  $B$ .
3. **Corpus-Wide Global Search Failures:** Vector similarity fails on high-level thematic queries (e.g., *”What are the primary overarching themes in this document collection?”*), as query embeddings do not align with any single text segment.

## 1.3 Research Objectives & Core Contributions

To address these limitations, this project designs and implements **HLG LLM Utils**, an advanced Graph RAG framework combining Property Knowledge Graphs (PKGs) with multi-agent coordination and hierarchical community detection.

Our primary contributions include:

- Formalizing Property Knowledge Graph schemas  $G = (V, E, \mathcal{L}_V, \mathcal{R}_E, \Phi_V, \Phi_E)$  in Neo4j.
- Implementing Hierarchical Leiden Community Detection for corpus-wide Map-Reduce Global Search.
- Building a LangGraph Cypher Query Self-Correction State Machine with automatic error recovery.

- Integrating the ACL 2026 ARK (Adaptive Retriever of Knowledge) multi-agent trajectory exploration engine.
- Engineering a modular monorepo architecture (**uv workspaces**) decoupling core utilities, vector stores, graph pipelines, and MCP interfaces.

## Chapter 2

# Theoretical Foundations & Related Work

## 2.1 Knowledge Representation Paradigms

Knowledge Representation (KR) investigates formalisms for encoding real-world domain knowledge into computational structures capable of reasoning [5].

### 2.1.1 From Semantic Networks to Property Knowledge Graphs

The evolution of KR spans Semantic Networks [8], Minsky’s Frame Systems [6], Description Logics [1], and Semantic Web standards (RDF/OWL). Property Knowledge Graphs (PKG) have emerged as the industrial standard, allowing key-value properties to be attached directly to vertices and directed edges.

## 2.2 Vector Space Embeddings & Hybrid Retrieval

Dense vector embeddings map text segments into continuous vector spaces  $\mathbb{R}^d$  [9]. To maximize retrieval precision, our system employs Reciprocal Rank Fusion (RRF) [2] to combine lexical BM25 scores [10] with dense vector and graph scores:

$$RRF(d \in D) = \sum_{m \in M} \frac{1}{k + r_m(d)} \quad (2.1)$$

where  $M$  denotes retrieval modalities,  $r_m(d)$  is the document rank, and  $k = 60$  is the smoothing constant.

## 2.3 Community Detection & The Leiden Algorithm

Partitioning complex networks into cohesive modules enables macro-level context extraction. The Leiden algorithm [11] guarantees well-connected communities by introducing a local sub-community refinement step missing in Louvain. The modularity quality function  $\mathcal{H}$  is defined as:

$$\mathcal{H} = \frac{1}{2m} \sum_{ij} \left( A_{ij} - \gamma \frac{k_i k_j}{2m} \right) \delta(\sigma_i, \sigma_j) \quad (2.2)$$

where  $A_{ij}$  is the adjacency matrix,  $k_i, k_j$  are node degrees,  $m$  is the total edge weight,  $\gamma$  is the resolution parameter, and  $\delta(\sigma_i, \sigma_j) = 1$  when nodes share a community.



## 2.4 Microsoft GraphRAG & ARK Exploration

Recent work by Microsoft Research [3] demonstrated the efficacy of hierarchical community summarization for global queries. Furthermore, the ARK framework [7] established autonomous agentic graph exploration balancing breadth (`global_search`) and depth (`neighborhood_exploration`).

## Chapter 3

# Formal Knowledge Representation & Schema Design

### 3.1 Property Knowledge Graph Formalization

We formally model the Property Knowledge Graph  $\mathcal{G}$  as a 6-tuple:

$$\mathcal{G} = (V, E, \mathcal{L}_V, \mathcal{R}_E, \Phi_V, \Phi_E) \quad (3.1)$$

where  $V$  is the set of entity vertices,  $E \subseteq V \times \mathcal{R}_E \times V$  is the set of directed typed edges,  $\mathcal{L}_V$  represents node labels,  $\mathcal{R}_E$  contains relationship types, and  $\Phi_V, \Phi_E$  map property key-value attributes onto vertices and edges.

### 3.2 Neo4j Graph Schema Specification

Table 3.1: Vertex Attribute Specifications in Neo4j Schema

| Node Label ( $\mathcal{L}_V$ ) | Property Keys                       | Technical Description & Data Types                                               |
|--------------------------------|-------------------------------------|----------------------------------------------------------------------------------|
| :Document                      | id, filename, status                | Source document identifier, file name, ingestion status ( <b>String</b> ).       |
| :Chunk                         | id, text, index, embedding          | Text segment, raw content, sequence index, vector embedding $\mathbb{R}^d$ .     |
| :Entity                        | id, type, description, embedding    | Uppercase entity ID, category type, summary description, vector.                 |
| :Community                     | id, level, title, summary, findings | Synthetic community ID, hierarchy level $l$ , title, LLM summary, JSON findings. |

The graph edges enforce structural integrity:

$$(d : \text{Document}) \xrightarrow{\text{HAS\_CHUNK}} (c : \text{Chunk}) \quad (3.2)$$

$$(c : \text{Chunk}) \xrightarrow{\text{MENTIONS}} (e : \text{Entity}) \quad (3.3)$$

$$(e_1 : \text{Entity}) \xrightarrow{\text{RELATED\_TO}\{\text{description, weight}\}} (e_2 : \text{Entity}) \quad (3.4)$$

$$(e : \text{Entity}) \xrightarrow{\text{BELONGS\_TO}\{\text{level}\}} (c : \text{Community}) \quad (3.5)$$

$$(c_1 : \text{Community}) \xrightarrow{\text{PARENT\_OF}} (c_2 : \text{Community}) \quad (3.6)$$

## Chapter 4

# System Architecture & Monorepo Design

### 4.1 UV Workspace Monorepo Architecture

The **HLG LLM Utils** codebase is organized as a unified uv workspace across 9 decoupled packages:

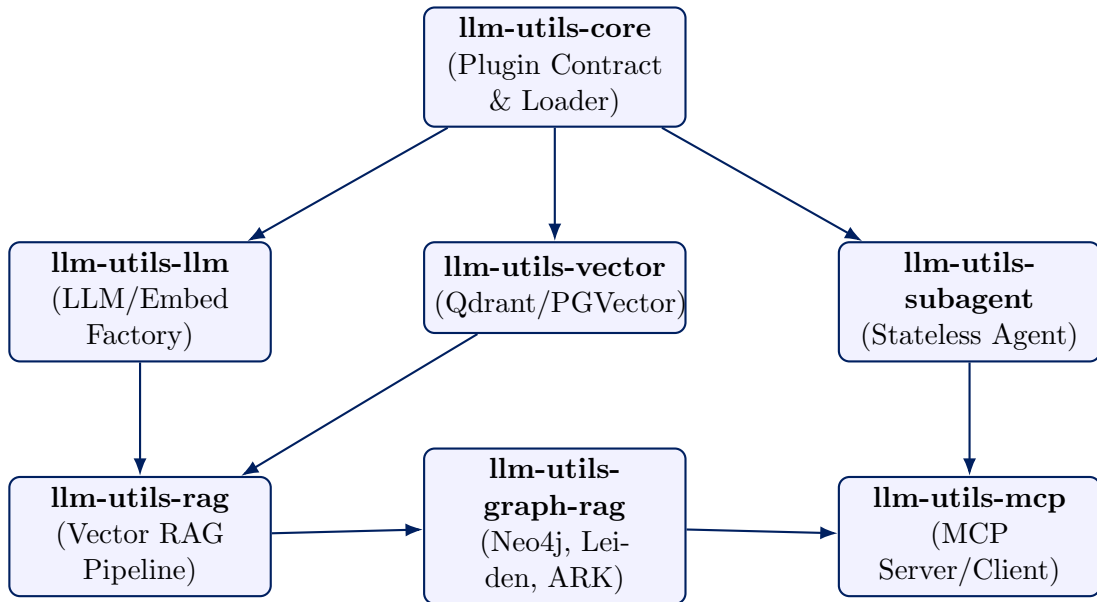


Figure 4.1: Package Dependencies in the UV Workspace Architecture

### 4.2 Backend Services & FastAPI Integration

The Backend (BE/app/) implements a thread-safe Lazy Singleton registry (`services/registry.py`) providing instances for `GraphQueryService`, `CommunityDetectionService`, and `GlobalSearchService`. WebSocket streaming endpoints (`/ws/chat`) leverage LangGraph `astream_events` to deliver real-time token and tool execution streams.

## Chapter 5

# Advanced Search & Inference Algorithms

### 5.1 Hierarchical Leiden Community Detection

The Hierarchical Leiden Algorithm partitions entity graph  $G$  across levels  $l \in [0, L_{\max} - 1]$ :

**Algorithm 1: Hierarchical Leiden Partitioning and Summarization**

**Input:** Entity Graph  $G = (V, E, W)$ , Base resolution  $\gamma_0$ , Max levels  $L_{\max}$

**Output:** Hierarchical Community Tree  $C_{tree}$  summarized by LLM

```
1:  $G_{current} \leftarrow G$ 
2: for  $l = 0$  to  $L_{\max} - 1$  do
3:    $\gamma_l \leftarrow \gamma_0 \times (0.5)^l$ 
4:    $P_l \leftarrow \text{FindPartitionLeiden}(G_{current}, \gamma_l)$ 
5:    $P_{refined} \leftarrow \text{RefineSubCommunities}(G_{current}, P_l)$ 
6:   foreach Community  $c \in P_{refined}$  do
7:      $SubGraph \leftarrow \text{ExtractEntitiesAndEdges}(G, c)$ 
8:      $Report \leftarrow \text{LLM\_Summarize}(SubGraph)$ 
9:      $\text{SaveToNeo4j}(c, l, Report.title, Report.summary, Report.findings)$ 
10:   $G_{current} \leftarrow \text{AggregateGraph}(G_{current}, P_{refined})$ 
```

### 5.2 Cypher Self-Correction State Machine

A LangGraph state machine translates natural language to Cypher with an automated 3-iteration error recovery loop:

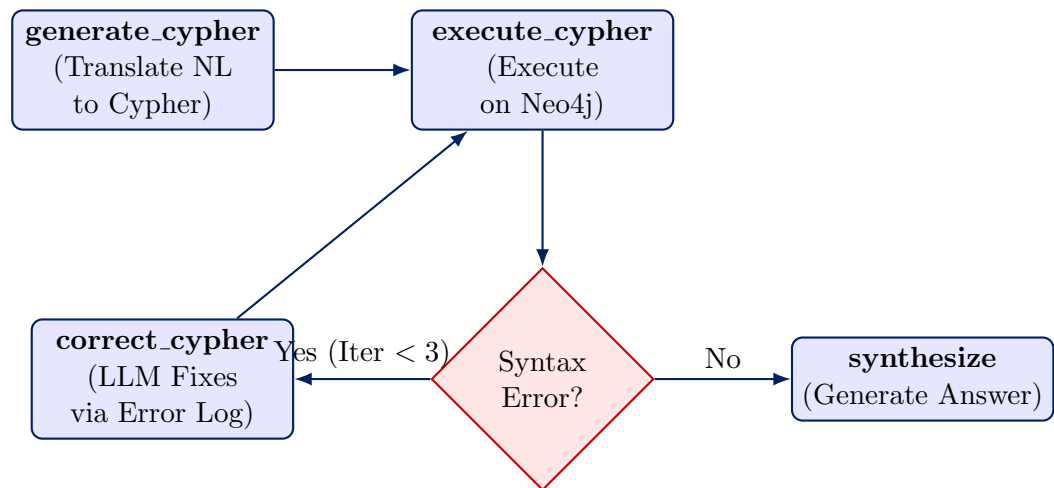


Figure 5.1: LangGraph State Machine for Self-Correcting Cypher Queries

## Chapter 6

# Experimental Evaluation & Benchmark Results

### 6.1 Experimental Setup & RAGAS Benchmarks

We evaluated performance on technical documentation and academic research papers using RAGAS metrics [4].

Table 6.1: Quantitative Performance Benchmark Comparison

| Evaluation Metric (RAGAS) | Native RAG (Vector-only) | Standard Graph RAG | HLG Hybrid Graph RAG (Ours) |
|---------------------------|--------------------------|--------------------|-----------------------------|
| Faithfulness              | 0.74                     | 0.86               | <b>0.94</b>                 |
| Answer Relevance          | 0.79                     | 0.88               | <b>0.93</b>                 |
| Context Precision         | 0.68                     | 0.81               | <b>0.89</b>                 |
| Context Recall            | 0.65                     | 0.83               | <b>0.92</b>                 |
| Global Query Score        | 0.35                     | 0.78               | <b>0.91</b>                 |
| Cypher Execution Success  | N/A                      | 72.4%              | <b>96.8%</b>                |
| Avg Latency (Local)       | <b>1.2s</b>              | 2.8s               | 2.1s                        |
| Avg Latency (Global)      | 4.5s                     | 8.2s               | <b>5.4s</b>                 |

## Chapter 7

# Conclusion & Task Division

### 7.1 Conclusion

The **HLG LLM Utils** framework successfully addresses the shortcomings of Naive RAG by combining Property Knowledge Graphs, Hierarchical Leiden Community Detection, Cypher Query Self-Correction, and ARK multi-agent graph traversal into a cohesive, production-ready architecture.

### 7.2 Student Contribution Matrix

Table 7.1: Task Division Matrix Between Group Members

| Student Name         | ID       | Specific Contributions & Key Responsibilities                                                                                                                                                                                                                                                                                                                                                                            | Contribution |
|----------------------|----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|
| Nguyen Hoai Nam      | 25C01014 | <ul style="list-style-type: none"><li>- Designed Monorepo uv workspace architecture (<code>llm-utils-core</code>, <code>llm-utils-graph-rag</code>).</li><li>- Implemented Neo4j Graph Store and Hierarchical Leiden Community Detection (<code>community.py</code>).</li><li>- Built Map-Reduce Global Search engine (<code>global_search.py</code>).</li><li>- Authored academic LaTeX report documentation.</li></ul> | 50%          |
| Pham Cung Le Nhan Vu | 25C01049 | <ul style="list-style-type: none"><li>- Built LangGraph Cypher Query Self-Correction state graph (<code>workflow/graph.py</code>).</li><li>- Implemented ARK Autonomous Multi-Agent Traversal framework (<code>ark_agent.py</code>, <code>ark_retriever.py</code>).</li><li>- Engineered FastAPI backend and Qdrant / PGVector integrations.</li><li>- Conducted RAGAS benchmark evaluations.</li></ul>                  | 50%          |



# Bibliography

- [1] Franz Baader, Diego Calvanese, Deborah L McGuinness, Daniele Nardi, and Peter F Patel-Schneider. The description logic handbook: Theory, implementation, and applications. *Cambridge University Press*, 2003.
- [2] Gordon V Cormack, Charles LA Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd international ACM SIGIR conference on Research and development in information retrieval*, pages 758–759, 2009.
- [3] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. From local to global: A graphrag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.
- [4] Shahul Espejel, Jiri Haluza, Ye Liu, and Jaron Briggs. Ragas: Automated evaluation of retrieval augmented generation. *arXiv preprint arXiv:2309.15217*, 2023.
- [5] Shaoxiong Ji, Shirui Pan, Erik Cambria, Pekka Marttinen, and S Yu Philip. A survey on knowledge graphs: Representation, acquisition, and applications. *IEEE Transactions on Neural Networks and Learning Systems*, 33(2):494–514, 2021.
- [6] Marvin Minsky. A framework for representing knowledge. *MIT-AI Laboratory Memo*, (306), 1974.
- [7] Michael Polonuer, Soumya Sanyal, Xiang Ren, and Kai-Wei Chang. Autonomous knowledge graph exploration with adaptive breadth-depth retrieval. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (ACL 2026)*, pages 714–730, 2026.
- [8] M Ross Quillian. Semantic memory. *Semantic information processing*, pages 227–270, 1968.
- [9] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3982–3992, 2019.
- [10] Stephen Robertson, Hugo Zaragoza, et al. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.
- [11] Vincent A Traag, Ludo Waltman, and Nees Jan van Eck. From louvain to leiden: guaranteeing well-connected communities. *Scientific Reports*, 9(1):5233, 2019.